

Cấu trúc dữ liệu

Cây nhị phân chỉ số (Binary Indexed Tree / Fenwick Tree)

Các bài toán truy vấn đoạn điển hình

- Range sum query:

Cho mảng A gồm N phần tử. Thực hiện Q truy vấn thuộc 2 loại:

1. Cộng x vào $A[u]$
2. Tính tổng các phần tử $A[0], A[1], \dots, A[p-1]$

Các bài toán truy vấn đoạn điển hình

- Range sum query:

Cho mảng A gồm N phần tử. Thực hiện Q truy vấn thuộc 2 loại:

1. Cộng x vào $A[u]$
2. Tính tổng các phần tử $A[0], A[1], \dots, A[p-1]$

- Ví dụ:

Có 3 truy vấn:

A	5	1	2	8	5	9	3	5
----------	---	---	---	---	---	---	---	---

2 4

1 4 -9

2 5

Các bài toán truy vấn đoạn điển hình

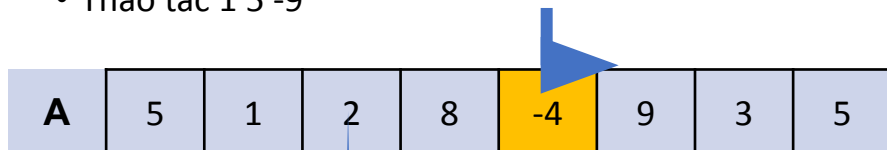
- Thao tác 2 4

$$5 + 1 + 2 + 8 = 16$$



- Thao tác 1 5 -9

$$5 + (-9)$$



- Thao tác 2 5

$$5 + 1 + 2 + 8 - 4 = 12$$



Các bài toán truy vấn đoạn điển hình

- Range sum query:

Cho mảng A gồm N phần tử. Thực hiện Q truy vấn thuộc 2 loại:

1. Cộng x vào A[u]
2. Tính tổng các phần tử A[0], A[1], ..., A[p-1]

```
void update(int u, int x) {  
    A[u] += x;  
}
```

```
void getSum(int p) {  
    int ans = 0;  
    for (int i = 0; i < p; ++i)  
        ans += A[i];  
    return ans;  
}
```

Các bài toán truy vấn đoạn điển hình

- Range sum query:

Cho mảng A gồm N phần tử. Thực hiện Q truy vấn thuộc 2 loại:

1. Cộng x vào A[u]
2. Tính tổng các phần tử A[0], A[1], ..., A[p-1]

```
void update(int u, int x) {  
    A[u] += x;  
}
```

ĐPT khi update: $O(1)$

```
void getSum(int p) {  
    int ans = 0;  
    for (int i = 0; i < p; ++i)  
        ans += A[i];  
    return ans;
```

ĐPT khi truy vấn: $O(p) \sim O(N)$

Các bài toán truy vấn đoạn điển hình

- Trong trường hợp nhiều truy vấn tính tổng?

Các bài toán truy vấn đoạn điển hình

- Trong trường hợp nhiều truy vấn tính tổng:

// Tiền xử lý

```
int createSumArr() {  
    sum[0] = A[0];  
    for (int i = 1; i < n; ++i) {  
        sum[i] = sum[i - 1] + A[i];  
    }  
}
```

```
void update(int u, int x) {  
    A[u] += x;  
    for (int i = u; i < n; ++i) {  
        sum[i] += x;  
    }  
}  
  
int getSum(int p) {  
    return sum[p];  
}
```

Các bài toán truy vấn đoạn điển hình

- Trong trường hợp nhiều truy vấn tính tổng:

```
// Tiền xử lý
int createSumArr() {
    sum[0] = A[0];
    for (int i = 1; i < n; ++i) {
        sum[i] = sum[i - 1] + A[i];
    }
}
```

ĐPT tiền xử lý: $O(N)$

```
void update(int u, int x) {
    A[u] += x;
    for (int i = u; i < n; ++i) {
        sum[i] += x;
    }
}

int getSum(int p) {
    return sum[p];
}
```

ĐPT khi update: $O(p) \sim O(N)$

ĐPT khi truy vấn: $O(1)$

Các bài toán truy vấn đoạn điển hình

- Vấn đề của cả 2 hướng tiếp cận trên:
 - Không cân bằng được độ phức tạp của thao tác cập nhật và thao tác tính tổng

Các bài toán truy vấn đoạn điển hình

- Vấn đề của cả 2 hướng tiếp cận trên:
 - Không cân bằng được độ phức tạp của thao tác cập nhật và thao tác tính tổng

=> Sử dụng BIT/Fenwick Tree

Binary Indexed Tree / Fenwick Tree

- Ý tưởng
 - Các số thập phân nào đều được biểu diễn bằng tổng các lũy thừa của 2.

Binary Indexed Tree / Fenwick Tree

- Ý tưởng
 - Các số thập phân nào đều được biểu diễn bằng tổng các lũy thừa của 2. Ví dụ:
 - $3 = 2^1 + 2^0$
 - $6 = 2^2 + 2^1$
 - $7 = 2^2 + 2^1 + 2^0$

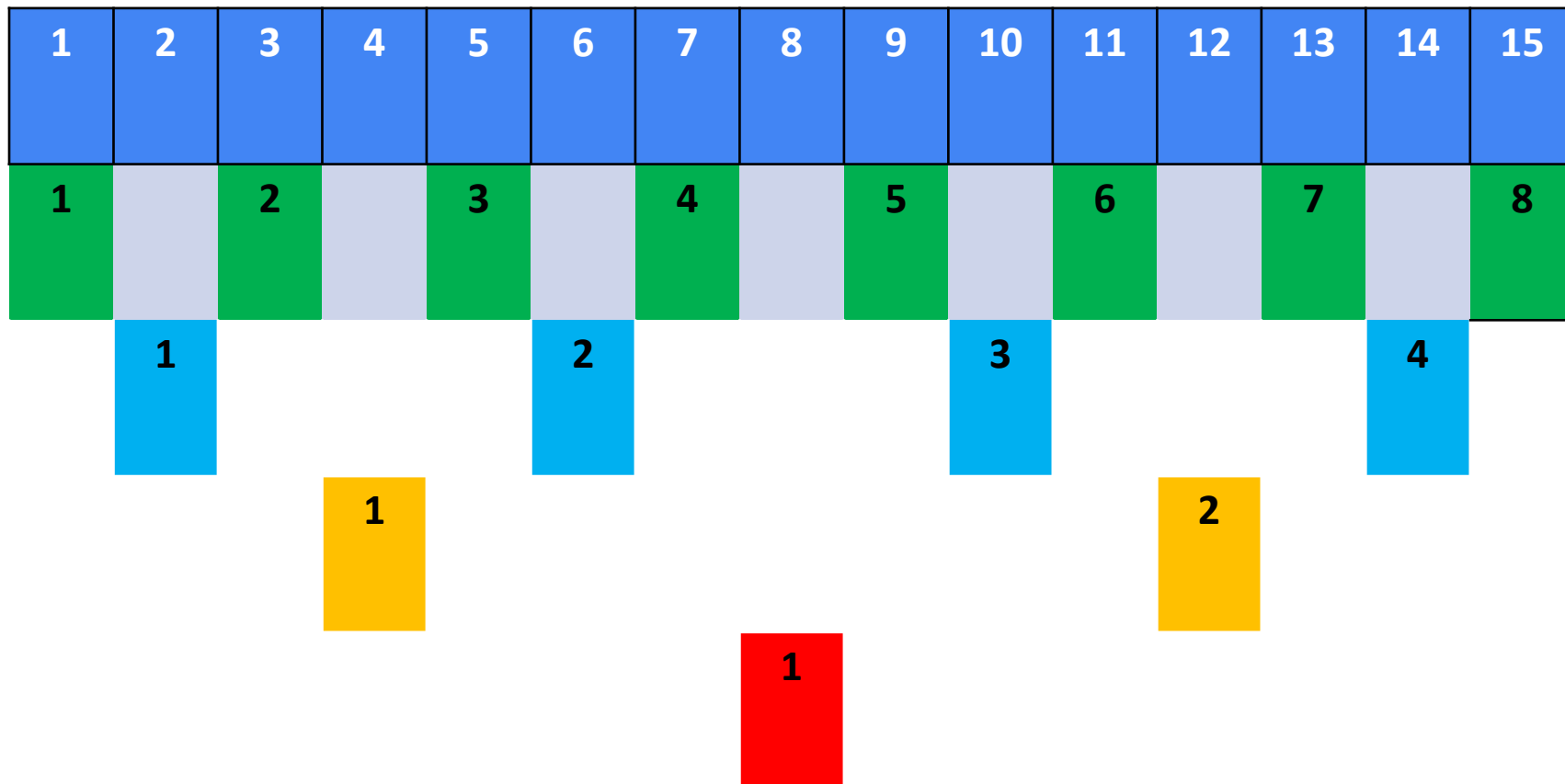
Binary Indexed Tree / Fenwick Tree

- Ý tưởng
 - Các số thập phân nào đều được biểu diễn bằng tổng các lũy thừa của 2. Ví dụ:
 - $3 = 2^1 + 2^0$
 - $6 = 2^2 + 2^1$
 - $7 = 2^2 + 2^1 + 2^0$
 - Để tính tổng của $[0, n-1]$, ta có thể tách thành từng đoạn có độ dài nhỏ hơn, mỗi đoạn có độ dài là 2^k

Binary Indexed Tree / Fenwick Tree

- Ý tưởng
 - Các số thập phân nào đều được biểu diễn bằng tổng các lũy thừa của 2. Ví dụ:
 - $3 = 2^1 + 2^0$
 - $6 = 2^2 + 2^1$
 - $7 = 2^2 + 2^1 + 2^0$
 - Để tính tổng của $[0, n-1]$, ta có thể tách thành từng đoạn có độ dài nhỏ hơn, mỗi đoạn có độ dài là 2^k
 - Ví dụ: $[0, 6] = [0, 3] + [4, 5] + [6, 6]$

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----



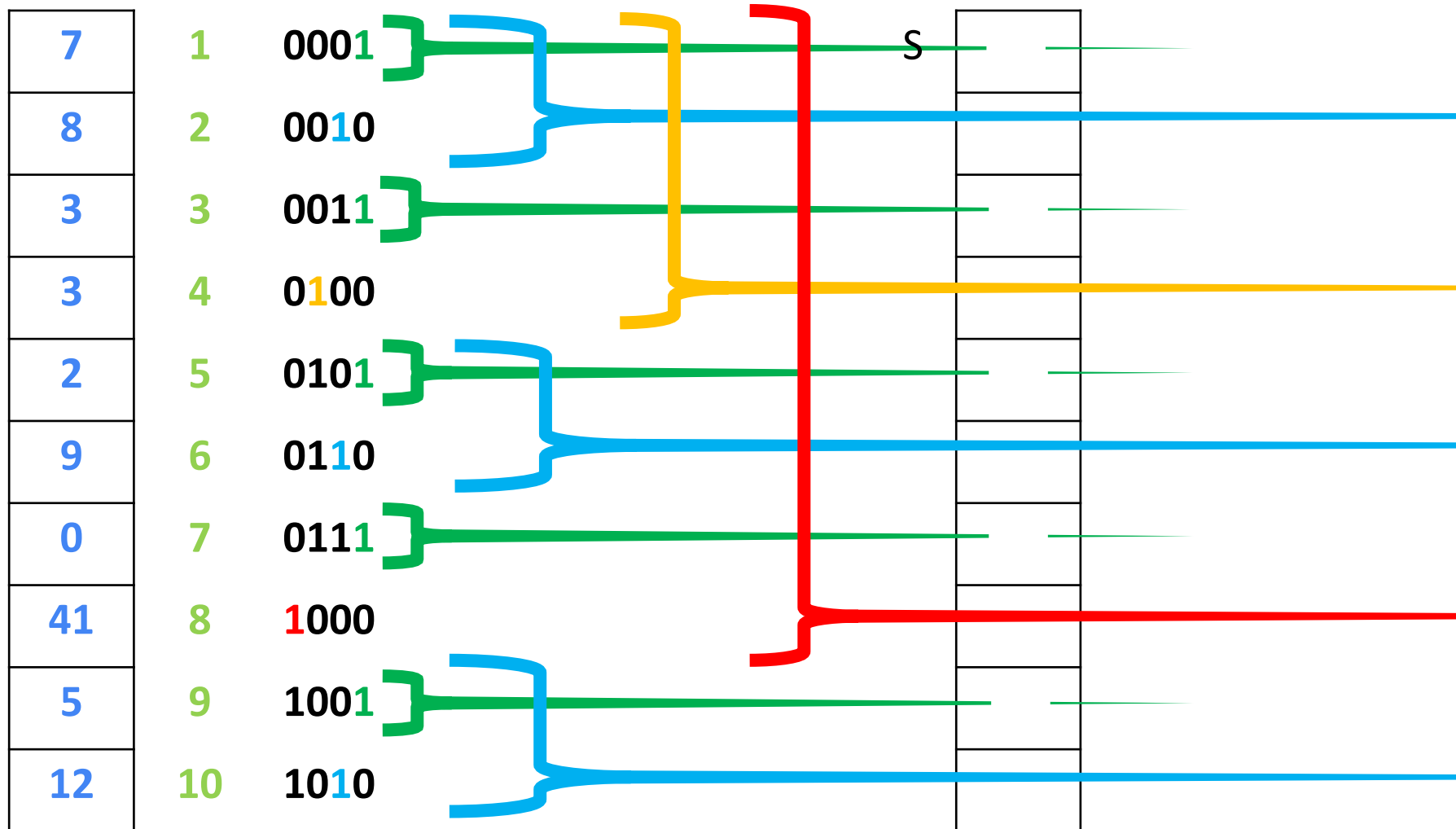
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

$$11 = 1011 = 2^3 + 2 + 1 = 8 + 2 + 1$$

Sum(11)=

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

A



A

7
8
3
3
2
9
0
41
5
12

8

S

7
15
3
21
2
11
0
73
5
17

Cặp nhật (3,5)

{

-

-

}

.

A

7
8
8
3
2
9
0
41
5
12

S

7
15
8
26
2
11
0
78
5
17

Sum(7)

{



Cài đặt

Lấy *giá trị* của bit 1 bên phải cùng:

$$n \ \& \ \sim(n - 1)$$

Ví dụ: $n = 10 = 1010_b$

$n =$	10 1 0
$n - 1 =$	10 0 1
$\sim(n - 1) =$	01 1 0
$\& n =$	10 1 0
$n \ \& \ \sim(n - 1) =$	00 1 0

Cài đặt

Sử dụng phép bù 2:

$$-n = \sim n + 1$$

$$\Rightarrow \sim n = -n - 1$$

Cài đặt

Sử dụng phép bù 2:

$$-n = \sim n + 1$$

$$\Rightarrow \sim n = -n - 1$$

Do đó

$$n \& \sim(n - 1) = n \& (-(n - 1) - 1) = n \& (-n)$$

Cài đặt

Lưu cây Fenwick: `int fen[N+1];`

Tìm tổng:

```
int getSum(int p) {  
    int ans = 0;  
    for (int i = p; i > 0; i -= (i & -i))  
        ans += fen[i];  
    return ans;  
}
```

Cài đặt

Lưu cây Fenwick: `int fen[N+1];`

Tìm tổng:

```
int getSum(int p) {  
    int ans = 0;  
    for (int i = p; i > 0; i -= (i & -i))  
        ans += fen[i];  
    return ans;  
}
```

=> Độ phức tạp: $O(\log N)$

Cài đặt

Cập nhật:

```
void update(int u, int v) {  
    for (int i = u; i <= N; i += (i & -i)) {  
        fen[i] += v;  
    }  
}
```

Cài đặt

Cập nhật:

```
void update(int u, int v) {  
    for (int i = u; i <= N; i += (i & -i)) {  
        fen[i] += v;  
    }  
}
```

=> Độ phức tạp: $O(\log N)$

Độ phức tạp

Tiền xử lý (đưa mảng A đề cho vào mảng fen):

Độ phức tạp

Tiền xử lý (đưa mảng A đề cho vào mảng fen): cập nhật N lần

Độ phức tạp

Tiền xử lý (đưa mảng A về mảng fen): cập nhật N lần

$$O(N \log N)$$

Độ phức tạp

Tiền xử lý (đưa mảng A đề cho vào mảng fen): cập nhật N lần

$$O(N \log N)$$

Thực hiện các truy vấn:

Độ phức tạp

Tiền xử lý (đưa mảng A đề cho vào mảng fen): cập nhật N lần

$$O(N \log N)$$

Thực hiện các truy vấn: $O(Q \log N)$

Độ phức tạp

Tiền xử lý (đưa mảng A đề cho vào mảng fen): cập nhật N lần

$$O(N \log N)$$

Thực hiện các truy vấn: $O(Q \log N)$

=> Độ phức tạp cả bài toán: **$O(N \log N + Q \log N)$**

Điểm mạnh & yếu

- Điểm mạnh:

- Dễ cài đặt
- Cân bằng độ phức tạp giữa thao tác tìm tổng và cập nhật

- Điểm yếu:

- Phụ thuộc vào tính chất của prefix sum
- Phụ thuộc vào số lượng và sự phân bố các chỉ số

Bài tập áp dụng

Dãy nghịch thế:

<https://oj.vnoi.info/problem/NKINV>

Bài tập áp dụng

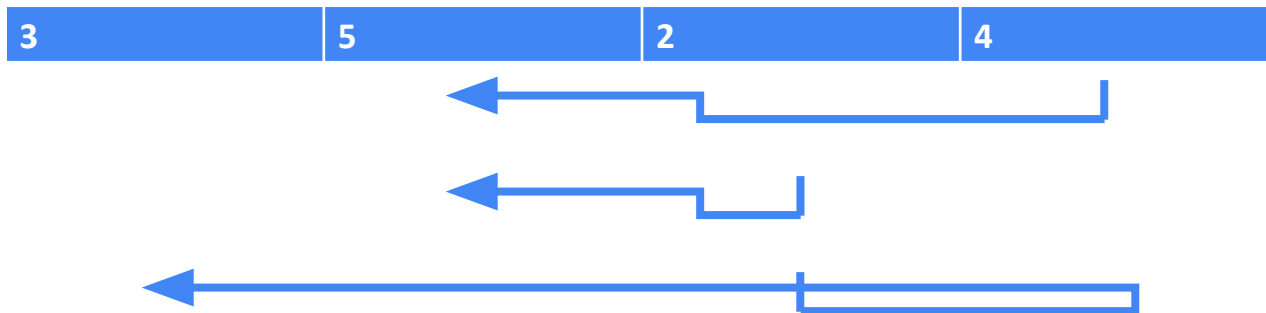
- Cho một dãy số $a_1 \dots a_N$. Một nghịch thế là một cặp số u, v sao cho $u < v$ và $a_u > a_v$. Nhiệm vụ của bạn là đếm số nghịch thế.
- Giới hạn
 - $1 \leq N \leq 60000$
 - $1 \leq a_i \leq 60000$

Nhận xét

- Tại một vị trí a_i bất kì, có k cặp nghịch thế (a_i, v)

- Ví dụ:

3 5 2 4



Ý tưởng sơ khai

- Sử dụng một mảng đánh dấu số lượng của số trong dãy, tại mỗi a_i ta xét tổng số nghịch thế dựa vào nhận xét, sau đó tổng tất cả lại ta được kết quả bài toán
- Ta có $C[a_i]$ là số lần xuất hiện của phần tử a_i nằm phía sau a_i
- Kết quả tại a_i là tổng số lần xuất hiện của các phần tử nhỏ hơn a_i

$$\text{Result} = \sum_1^{a[i]-1} C[x]$$

● Ví dụ: 3 5 2 4

- $A_4 = 4$, $\text{Result} += \sum_1^3 C[i] \text{ (=0)}$
 - $S[4] += 1$
 - $A_3 = 2$, $\text{Result} += \sum_1^1 C[i] \text{ (=0)}$
 - $S[2] += 1$
 - $A_2 = 5$, $\text{Result} += \sum_1^4 C[i] \text{ (=2)}$
 - $S[5] += 1$
 - $A_1 = 3$, $\text{Result} += \sum_1^2 C[i] \text{ (=1)}$
- ➔ Tổng số nghịch thế = 2 + 1 = 3

Nhận xét ý tưởng sơ khai

- Các bước tính tổng từ 1 đến a_i trong mảng C tốn một thao tác $O(N)$
- Thao tác cập nhật tốn $O(1)$
- Tuy nhiên, với những bài toán tính tổng dãy, có thể tối ưu thao tác này về $O(\log N)$ bằng Fenwick Tree (BIT)

Ý tưởng dùng BIT

- Thao tác `getbit(a[i])` trên mảng
- Thao tác cập nhật
Cập nhật $C[a[i]] += 1$ bằng thao tác `setbit(a[i])`

Ý tưởng dùng BIT

- Thao tác getbit(a[i]) trên mảng
- Thao tác cập nhật
Cập nhật $C[a[i]] += 1$ bằng thao tác setbit(a[i])

```
int getbit(int k){  
    int Result=0;  
    while(k>0){  
        Result+=C[k];  
        k-=(k&(-k));  
    }  
    return Result;  
}
```

```
void setbit(int k){  
    while (k<=60000){  
        C[k]+=1;  
        k+=(k&(-k));  
    }  
}
```

Thuật toán hoàn chỉnh

```
int main() {  
    cin>>n;  
    for (int i=1;i<=n;i++){  
        cin>>a[i];  
    }  
    for (int i=n;i>=1;i--){  
        Res+=getbit(a[i]);  
        setbit(a[i]+1);  
    }  
    cout<<Res;  
    return 0;  
}
```

```
int getbit(int k){  
    int Result=0;  
    while(k>0){  
        Result+=C[k];  
        k-=(k&(-k));  
    }  
    return Result;  
}
```

```
void setbit(int k){  
    while (k<=60000){  
        C[k]+=1;  
        k+=(k&(-k));  
    }  
}
```

Bài tập áp dụng

Cho mảng A gồm n số nguyên dương ($n \leq 10^6$, $a[i] \leq 10^9$)

- $f(L, R, x)$: số phần tử của mảng A trong đoạn từ L đến R có giá trị bằng x.

Yêu cầu: Đếm số cặp (i, j) sao cho $f(1, i, a[i]) > f(j, n, a[j])$, $i < j$.

Ví dụ:

Input	Output
7 1 2 1 1 2 2 1	8
3 1 1 1	1
5 1 2 3 4 5	0